Linked List

Structures

It is a data type used for handling a group of logically related data items.
For example: record of student : name, roll number, marks

```
struct  personal
{
      char    name[20];
      int     day;
      char    month[10];
      int     year;
      float   salary;
};
main()
{
      struct personal person;

      printf("Input Values\n");
      scanf("%s %d %s %d %f",
                person.name,
                &person.day,
                person.month,
                &person.year,
                &person.salary);
      printf("%s %d %s %d %f\n",
                person.name,
                person.day,
                person.month,
                person.year,
                person.salary);
   }
```

```
#include <stdio.h>
struct employee
                {
int emp_id;
                char emp_name[20];
                } e1;
int main()
{
struct employee *e;
printf("enter the details of employee1\n");

scanf("%d%s",&e1.emp_id,e1.emp_name);
printf("employees details\nemployee id
=%d\nemployee name =
%s",e1.emp_id,e1.emp_name);
e=&e1;
printf("\n%d%s",e->emp_id,e->emp_name);
                return 0;
}
```

Self Referential Structure

A self-referential structure is a structure that can have members which point to a structure
variable of the same type

```
struct node
{
int data;
struct node *next;
};
```

Linked List

- A linked list is a data structure which allows to store data dynamically and manage data efficiently.
- Linked List can be defined as collection of objects called **nodes** that are randomly stored in the memory.
- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the null.
- There are three common types of Linked List.

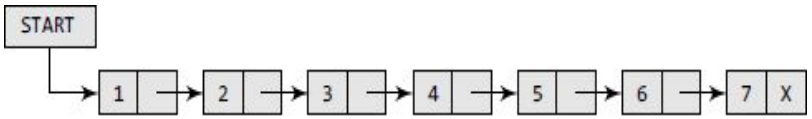


Figure 6.1 Simple linked list

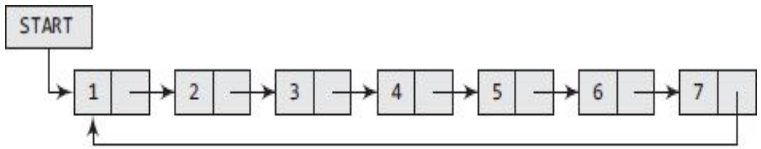


Figure 6.26 Circular linked list

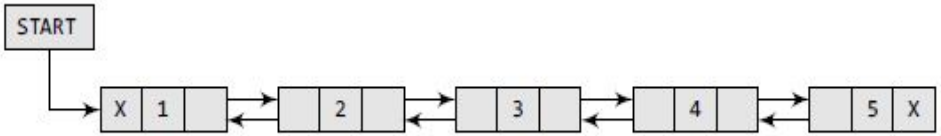


Figure 6.37 Doubly linked list

1.Singly Linked List

2.Doubly Linked List

3.Circular Linked List

Linked list as Abstract data type

- Linked List is an Abstract Data Type (ADT) that holds a collection of Nodes, the nodes can be accessed in a sequential way. Linked List doesn't provide a random access to a Node.

- Usually, those Nodes are connected to the next node and/or with the previous one, this gives the linked effect. When the Nodes are connected with only the next pointer the list is called Singly Linked List and when it's connected by the next and previous the list is called Doubly Linked List.
- Perform following operations

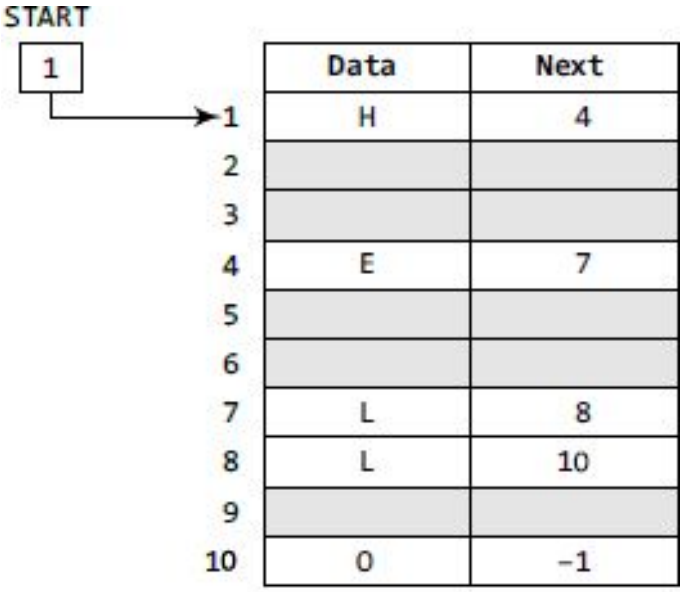
Insertion at the beginning, end, at particular position

Deletion at the beginning, end, at particular position

Traversal

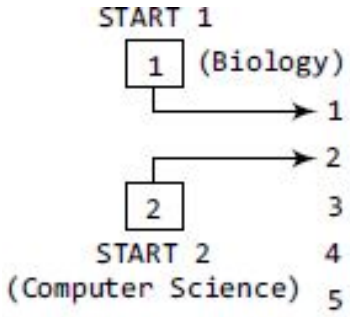
Reversal

Memory Representation of Linked List



	Data	Next
1	H	4
2		
3		
4	E	7
5		
6		
7	L	8
8	L	10
9		
10	O	−1

Figure 6.2 START pointing to the first element of the linked list in the memory



	Roll No	Next
1	S01	3
2	S02	5
3	S03	8
4		
5	S04	7
6		
7	S05	10
8	S06	11
9		
10	S07	12
11	S08	13
12	S09	−1
13	S10	15
14		
15	S11	−1

Figure 6.3 Two linked lists which are simultaneously maintained in the memory

Arrays Vs Linked List

ARRAY	LINKED LISTS
1. Arrays are stored in contiguous location.	1. Linked lists are not stored in contiguous location.
2. Fixed in size.	2. Dynamic in size.
3. Memory is allocated at compile time.	3. Memory is allocated at run time.
4. Uses less memory than linked lists.	4. Uses more memory because it stores both data and the address of next node.
5. Elements can be accessed easily.	5. Element accessing requires the traversal of whole linked list.
6. Insertion and deletion operation takes time.	6. Insertion and deletion operation is faster.

Array vs Linked List

Free Pool/ Avail List

list of available space is called the *free pool/ Avail list*

START → 1 (Biology)
4 AVAIL

	Roll No	Marks	Next
1	S01	78	2
2	S02	84	3
3	S03	45	5
4			6
5	S04	98	7
6			9
7	S05	55	8
8	S06	34	10
9			14
10	S07	90	11
11	S08	87	12
12	S09	86	13
13	S10	67	15
14			-1
15	S11	56	-1

Figure 6.6 Linked list with AVAIL and START pointers

START → 1 (Biology)

	Roll No	Marks	Next
1	S01	78	2
2	S02	84	3
3	S03	45	5
4			
5	S04	98	7
6			
7	S05	55	8
8	S06	34	10
9			
10	S07	90	11
11	S08	87	12
12	S09	86	13
13	S10	67	15
14			
15	S11	56	-1

(a)

START → 1 (Biology)

	Roll No	Marks	Next
1	S01	78	2
2	S02	84	3
3	S03	45	5
4	S12	75	-1
5	S04	98	7
6			
7	S05	55	8
8	S06	34	10
9			
10	S07	90	11
11	S08	87	12
12	S09	86	13
13	S10	67	15
14			
15	S11	56	4

(b)

Figure 6.5 (a) Students' linked list and (b) linked list after the insertion of new student's record

Garbage Collection: The operating system scans through all the memory cells and marks those cells that are being used by some program. Then it collects all the cells which are not being used and adds their address to the free pool, so that these cells can be reused by other programs. This process is called *garbage collection*.

Singly Linked list

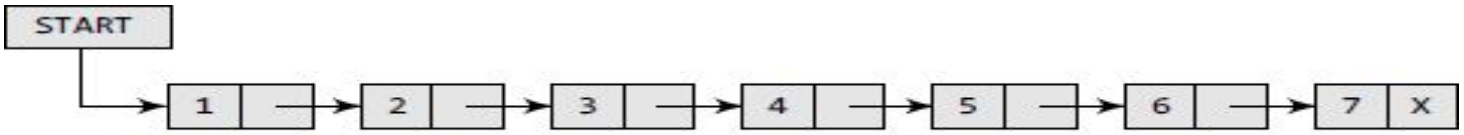
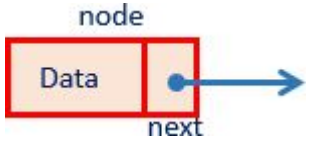


Figure 6.7 Singly linked list

- A start pointer stores address of the next node.
- Each node contains two parts information and link(pointer to next node)
- Link of the last node is NULL (indicates end of linked list).

```
struct node
{
int data;
struct node *next;
};
```



Note:
Such structures which contain a member field pointing to the same structure type are called self-referential structures.

Operations on Linked

- Traversing a list
  - Printing, finding minimum, etc.
- Insertion of a node into a list
  - At front, end and anywhere, etc.
- Deletion of a node from a list
  - At front, end and anywhere, etc.
- Comparing two linked lists
  - Similarity, intersection, etc.
- Merging two linked lists into a larger list
  - Union, concatenation, etc.
- Ordering a list
  - Reversing, sorting, etc.

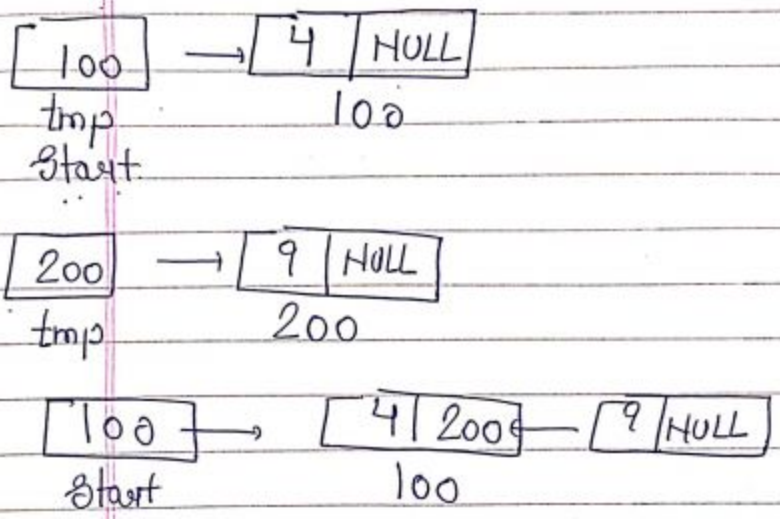
Creation of Linked List

```
struct node
{
   int info;
   struct node *link;
} *start;

void create_List (int data)
{
   struct node *q, *tmp;
   tmp = (struct node*) malloc (size of (struct node));
   tmp -> info = data;
   tmp -> link = NULL;
   if (start == NULL):
   {
      start = tmp;
   }
```

Creation of Linked List

```
else {
    q = start;
    while (q->link != NULL)
        q = q->link;
    q->link = tmp;
}
```

Time Complexity create a list = O(n)



Traversal of Linked List

```
void display()
{
  struct node *q;
 if (start == NULL)
  {
   printf("\n list is empty");
      return;
  }
  q = start;
  printf("\n list is");
  while(q != NULL)
   {
     printf("%d", q->info);
      q = q->link;
   }
   printf("\n");
}   Time Complexity of display = O(n)
```

Searching in a Linked

```
Void Search (int data)
{
  Struct node *ptr = start;
   int pos = 1;

  while(ptr! = NULL)
  { if ( ptr→ info = = data)
    {
    printf ("\n\n Item%ds found at position :%d", data,
                                                  pos.);

      return;
    }
      ptr = ptr→ link;
       pos++;
  }
    if (ptr = = NULL)
      printf (" \n Item %d is not found", data);
  }}

Time Complexity Search = O(n)
```

Insertion in linked list

Case 1: The new node is inserted at the beginning.
Case 2: The new node is inserted at the end.
Case 3: The new node is inserted after a given node.

Overflow is a condition that occurs when no free memory cell is present in the system.

Insertion at the beginning

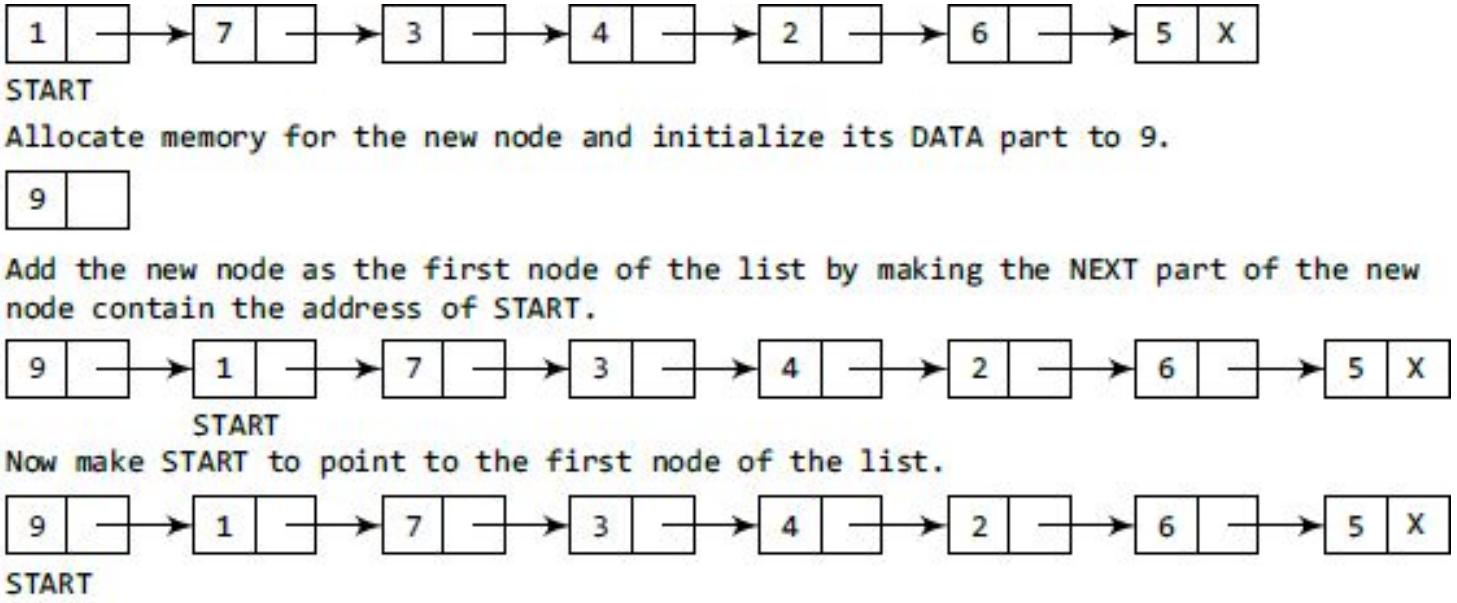


Figure 6.12 Inserting an element at the beginning of a linked list

Insertion at the beginning

```
void Insert at Beg (int data)
{
    struct node *tmp;
  tmp = (struct node*) *malloc( Size of (struct node))
      tmp→ info = data;
      tmp → link = start;
            start = tmp;
}
```

Time complexity insert at beg = O(1)

Insertion at the end of the linked list

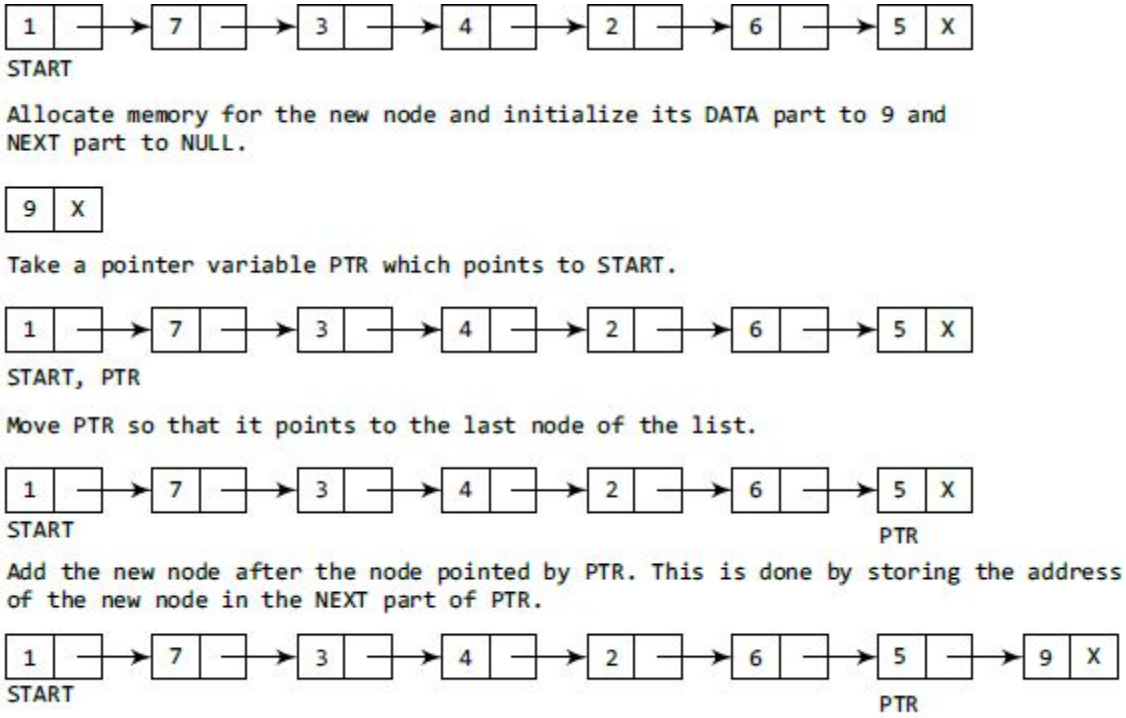


Figure 6.14 Inserting an element at the end of a linked list

Insertion at the end of the linked list

```
Void insert at end (int data)
{
   struct node * tmp, *q;
   tmp = (struct node*) malloc ( size of (struct node) );
   tmp → info = data;
   tmp → link = NULL;
   q = start;
   while (q → link != NULL)
   {
      q = q → link;
   }
   q → link = tmp;
}
```

Time Complexity Insert at end = O(n)

Insertion after a given node

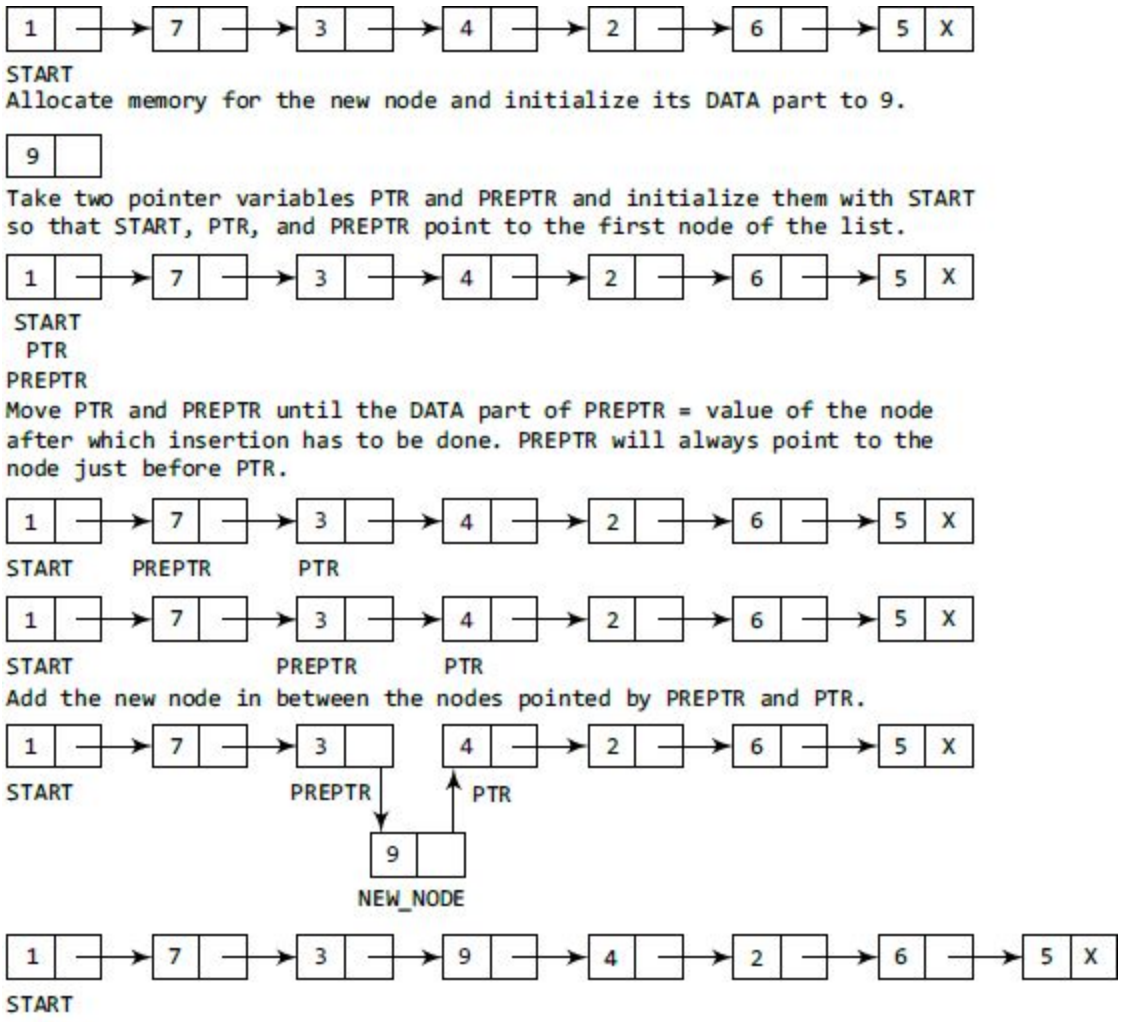


Figure 6.17 Inserting an element after a given node in a linked list

Insertion after a given node

```
Void insert After pos (int data)
{
  struct node *tmp, *q;
  int i;
  q = start;
  for( i=0; i<pos-1; i++ )
  { q = q->link;
```

```
if (q==NULL)
{ printf("\n Invalid position");
  exit getch(); return;
}

tmp = (struct node*) malloc (sizeof (struct node));
  tmp->info = data;
  tmp->link = q->link;
  q->link = tmp;
}
```

Time Complexity Insert at pos = O(n)

Deletion of a node from linked list

Case 1: The first node is deleted. Case 2: The last node is deleted.

Case 3: The node after a given node is deleted.

Underflow: when linked list is empty no deletion is possible such condition is known as underflow

Free Memory after deletion

- Do not forget to free() memory location dynamically allocated for a node after deletion of that node.
- It is the programmer's responsibility to free that memory block.
- Failure to do so may create a dangling pointer – a memory, that is not used either by the programmer or by the system.
- The content of a free memory is not erased until it is overwritten.

Deletion from first node

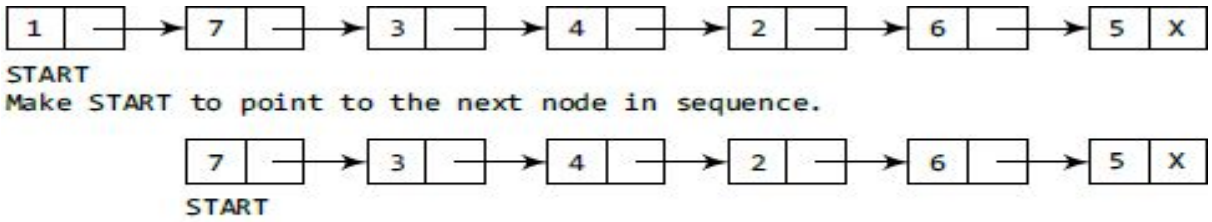


Figure 6.20 Deleting the first node of a linked list

```
void deleteFromBeg()
{
  struct node *tmp;
  tmp = start;
  if (start == NULL)
  printf ("list is empty");
  else
  {
    tmp = start;
    start = start->next;
    free(tmp);
  }
}
```

Deletion of last node

```
void DelekfromEnd()
{
  struct node *prevnode, *tmp;
    tmp = start;
  while (tmp -> link != NULL)
    {
      prevnode = tmp;
    } tmp = tmp->next;
```

```
if (tmp == start)
    start = NULL;
else
    Prevnode -> next = NULL;
        free(tmp);
}

Time Complexity Delek from End = O(n)
```

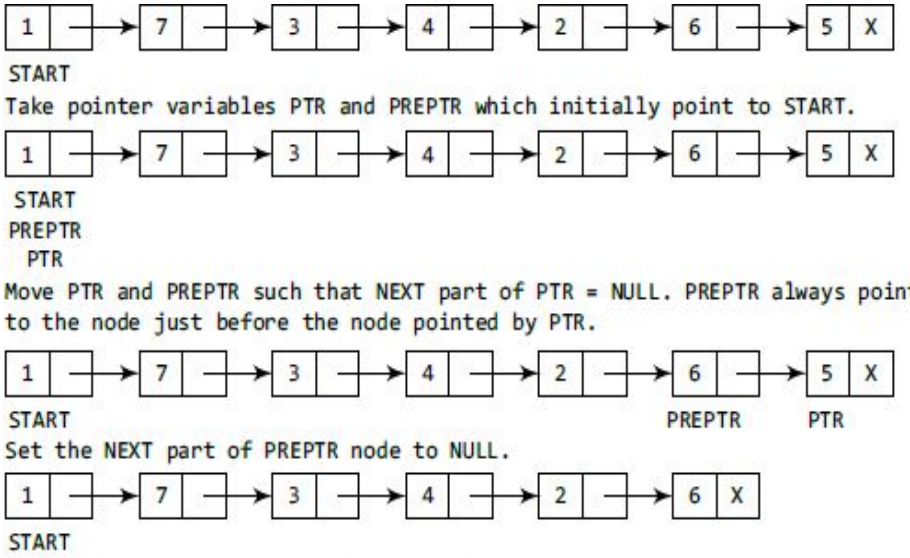


Figure 6.22 Deleting the last node of a linked list

Delete a node after given node

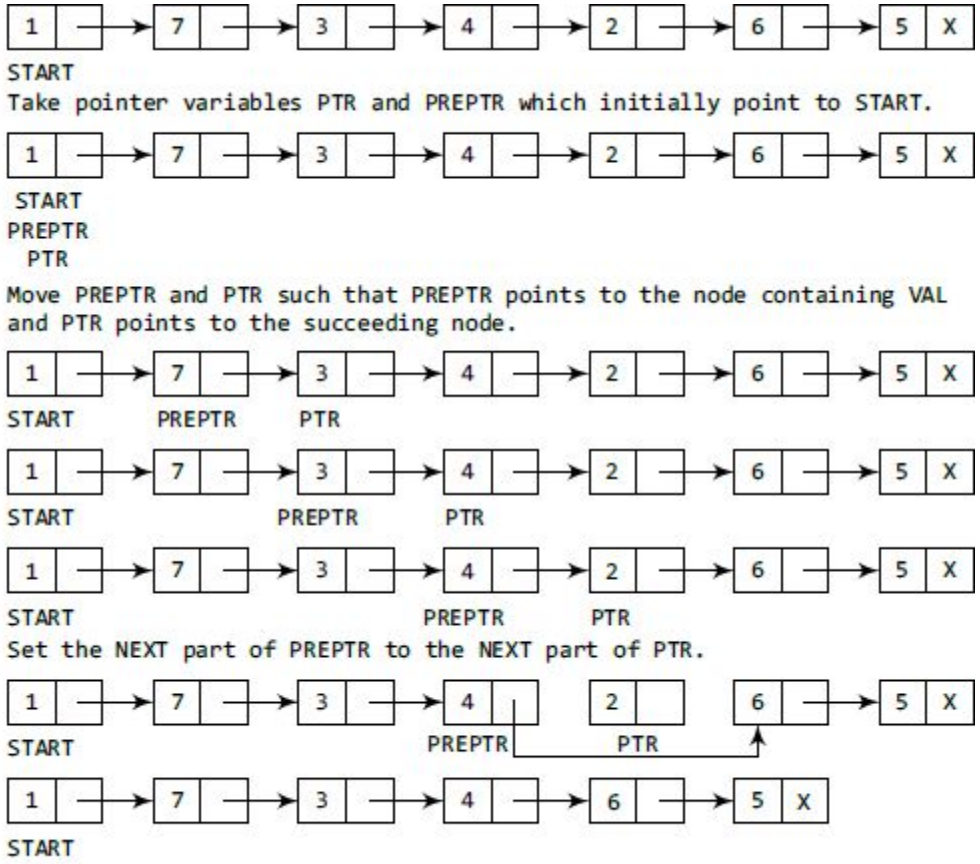


Figure 6.24 Deleting the node after a given node in a linked list

Delete a node after given

```
void deleteafterpos (int pos)
{
   struct node * ptr, *ptr1;
   ptr = start;
   for(int i = 0; i<pos; i++)
   {
     ptr1 = ptr;
     ptr = ptr -> next;
     if (ptr == NULL)
     {
       printf ("Invalid position");
       return;
     }
     ptr1 -> next = ptr -> next;
     free(ptr);
     printf ("\n Deleted %node", loc);
   }
```

Time Complexity deletion after pos = O(n)

Doubly linked list

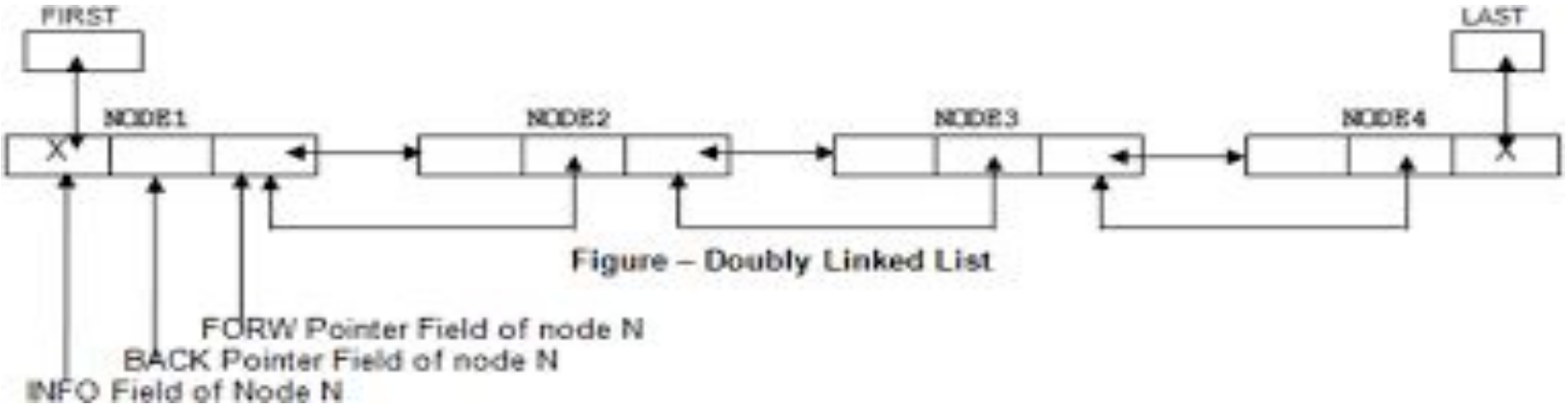


Figure – Doubly Linked List

```
struct node
{
        struct node *prev;
        int data;
        struct node *next;
};
```

Doubly linked list VS singly linked list

Advantages over singly linked list

1) A DLL can be traversed in both forward and backward direction.
2) The delete operation in DLL is more efficient if pointer to the node to be deleted is given.

Disadvantages over singly linked list

1) Every node of DLL Require extra space for an previous pointer.
2) All operations require an extra pointer previous to be maintained.

Operations on Doubly linked list

Insertion at beginning Insertion at end
Insertion after given position
Traversal
Deletion at the beginning Deletion at the end
Delete a node

Creation of Doubly Linked List

```
struct node
{
  int info;
  struct node *prev;
  struct node * next;
} * start;

void create_list (int x)
  { struct node *tmp,q;
    tmp = (struct node *) malloc ( sizeof(structnode));
    tmp → info = x;
    tmp → prev = null;
    tmp → next = null;

  if (start = = null)
    {
      tmp→prev = null;
      start→prev = tmp;
        start = tmp;
    }
  else
      { q = start;
```

```
while(q→next != HOLL)
  { q = q→next;
  }
    q→next = tmp;
     tmp→prev = q;
}
```

Time Complexity of create Double linked list = O(n)

Insertion at Beginning

```
void addatbeg(int x)
{ struct node *tmp;
  tmp = (struct node *) malloc (sizeof(struct node));
  tmp→info = x;
  tmp→prev = null;
  tmp→next = null;

   if (start == =null)
   {
       start = tmp;
   }
   else
     {
       tmp→next = start;
       start→prev = tmp;
         start = tmp;
   }
```

Time Complexity insert at beg of the ~~linke~~ doubly linked list = O(1)

Insertion at End

```
void insertatend (int x)
 {
    struct node *tmp, *q;
 tmp = (struct node*) malloc (sizeof (struct node));
tmp→ info = x;
tmp→ prev = null;
tmp→ next = null;
 if (start == HULL)
    start = tmp;
else
 {
    q = start;
    while (q→ next! = HULL)
      {
        q = q→ next;
      }
      q → next = tmp;
      tmp→ prev = q;
    }}
```

Time Complexity insert at end = O(n)

Insertion after Position

```
void addafterpos (int num, int pos)
{
  struct node *tmp, *prev;
  int i;
  struct node *q = start;
  for(i=0; i<pos-1; i++)
  {
    q = q->next;
    if(q == NULL)
    {
      printf("Invalid position");
      return;
    }
  tmp = (struct node *)malloc(sizeof(struct node));
  tmp->info = num;
  q->next->prev = tmp;
  tmp->next = q->next;
  tmp->prev = q;
  q->next = tmp;
```

Time Complexity Insert at pos = O(n)

Traversal

```
void display()
{
  struct node *q;
  if (start == HULL)
  printf ("list is empty");
  else
  {
    q = start;
   while (q != HULL)
   {
    printf ("%d", q→info);
     q = q→next;
   }
  }
}
```

Deletion at Beginning

```
void deletefrombeg()
{ struct node *tmp = start;
  if (start == NULL)
{ printf("list is empty");
  return;
}
```

Time Complexity for deletion at beg = O(1)

```
else if (tmp → next == NULL)
  {
      start = NULL;
    return;
  }
else
  {
    tmp = start;
    start = start → next;
    start → prev = NULL;      free(tmp);
}}
```

Deletion at End

```
Void deleteFromEnd()
{
  struct node *tmp = start;
  if(start == NULL)
  {
    printf("List is empty");
    return;
  }
  else if (tmp -> next == NULL)
  {
    start = NULL;
    return;
  }
  while(tmp -> next != NULL)
  {
    tmp = tmp -> next;
  }
  struct node *q = tmp -> prev;
  q -> next = NULL;
  free(tmp);
}
```

Time Complexity for deletion at end = O(n).

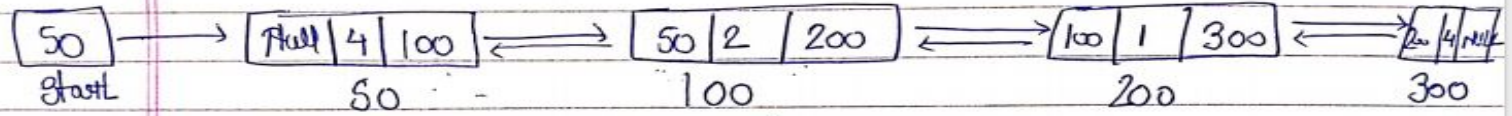
Deletion after Position

```
void delafterpos(int pos)
{
  int pos, i = 0;
  struct node * temp, = start;
  printf"
  while (i<pos)
  {
    temp = temp -> next;
    i++;
```

```
  if (temp = = NULL)
  {
    printf ("Invalid position return");
    return;
  }
}
```

Deletion after Position

```
temp → prev → next = temp → next;
temp → next → prev = temp → prev;
free(temp);
}
```



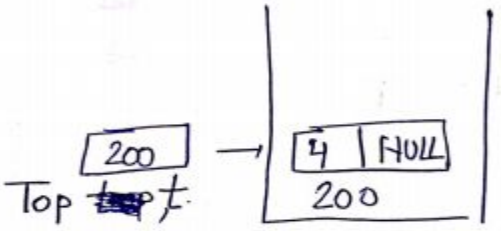
Pos = 1 , temp = 50

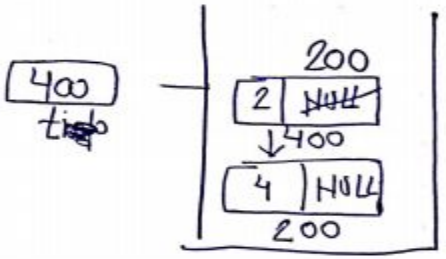
i	temp	i < pos	temp
0	~~50~~	T	100
1		F	

```
temp → prev → next = temp → next;
temp → next → prev = temp → prev;
free(temp);
}
```

Stack Using Linked List

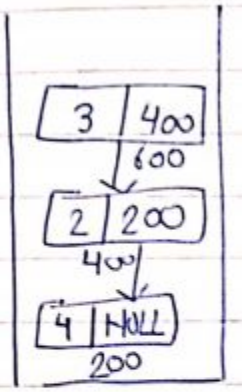
```
struct stack
{
   int data;
   struct stack *link;
};
struct stack * Top = NULL;
void push (int x)
{ struct stack *t;
 t = (struct stack*) malloc( sizeof(struct stack));
 t→ data =x;
t→ link = NULL;
      if (Top = = NULL )
            Top =t;

       else
         {
        t→ next = Top;
        Top = t;
    }
}
```





Stack Using Linked List

```
Void pop()
{   struct stack *p;
   if (Top == NULL)
{ printf("Stack is Underflow");
        return;
}
   else
   {   p = Top;
      Top = Top → next;
        free(p);
   }
}
```



Stack Using Linked List

```
void display()
{
  struct stack *p;
 if (Top == NULL)
 {
  Printf("Stack is underflow");
        return;
  }
 else {
        p = Top;
        while(p!=NULL)
        {
        Printf("%d", p->data);
          p = p->next;
        }
  }
}
```

Time Complexity of Push operation = O(1)

Time Complexity of Pop operation = O(1)

Time Complexity of display operation = O(n)

Stack Using Linked List

```
        {
    printf("%d", p→data);
        p = p→next;
  }
 }
}
```

Time Complexity of Push operation = O(1)

Time Complexity of Pop operation = O(1)

Time Complexity of display operation = O(n)

Queue Using Linked List

Queue using linked list

```
struct que
{ int data;
  struct que * next;
} struct que * f = NULL; struct que * r = NULL;
```

Queue Using Linked List

```
Void enqueue (int x)
{  struct que *t;
   if (f == NULL && r == NULL)
      f = r =
   t = (struct que *) malloc ( Size of (struct que));
   t → data = x;
   t → next = NULL;
   if (f == NULL && r == NULL)
       f = r = t;
   else
   {
       r → next = t;
          r   = t;
   }
}
```

Time Complexity of enqueue operation = O(1)

Queue Using Linked List

```
void dequeue()
{
  struct node *t;
    if(f==HULL && r==HULL)
    { printf("Underflow");
        return;
    }
```

```
else
  {
    t=f;
    f=f->next;
    free(f);
  }
}
```

Time Complexity of dequeue operation = O(1)

Queue Using Linked List

```
void display()
{
 struct node *t;
 if(f==NULL && r==NULL)
 {
  printf("queue is empty");
     return;
 }
 else
   { struct node *q=f;
        while(q!=NULL)
        { Printf("%d",q->data);
          q=q->next;
        }
   }
```

Circular Linked List

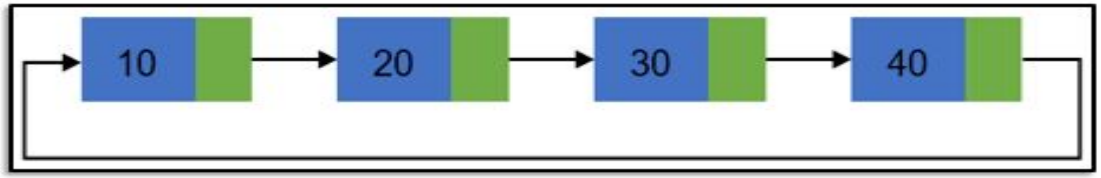
- The pointer from the last element in the list points back to the first element.
- A circular linked list is basically a linear linked list that may be single- or double-linked.

- The only difference is that there is no any NULL value terminating the list.
- In fact in the list every node points to the next node and last node points to the first node, thus forming a circle. Since it forms a circle with no end to stop it is called as **circular linked list**.

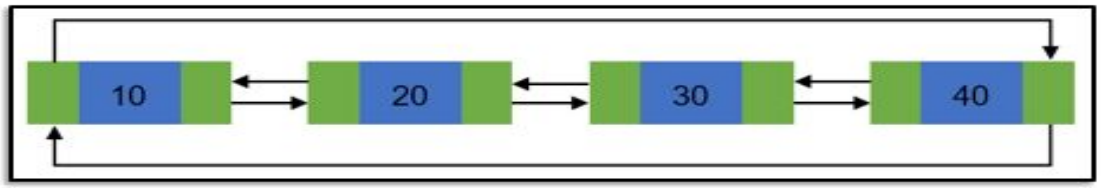
- In circular linked list there can be no starting or ending node, whole node can be traversed from any node.

- In order to traverse the circular linked list, only once we need to traverse entire list until the starting node is not traversed again.

Basic structure of singly circular linked list:



Doubly circular linked list:



Advantages and Disadvantages of Circular Linked List

Advantages of a Circular linked list

- Entire list can be traversed from any node.
- Circular lists are the required data structure when we want a list to be accessed in a circle or loop.
- Despite of being singly circular linked list we can easily traverse to its previous node, which is not possible in singly linked list.

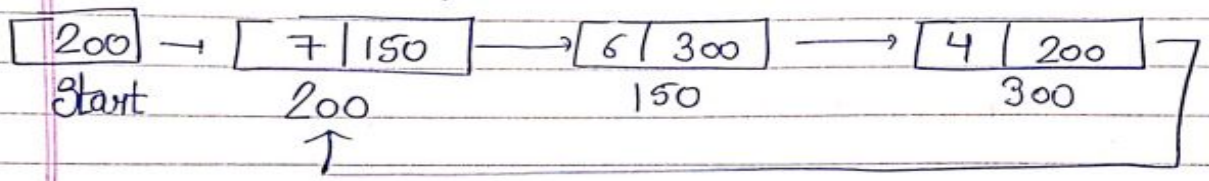
Disadvantages of Circular linked list

- Circular list are complex as compared to singly linked lists.
- Reversing of circular list is a complex as compared to singly or doubly lists.
- If not traversed carefully, then we could end up in an infinite loop.

VIPS Vivekananda Institute of Professional Studies
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

Creation of Circular Linked List

Implementation of circular linked list.




```
~~Void Be create(int~~
Struct node
  {
    int data;
    struct node * next;
  } * start = NULL;

Void Create (int x)
  {
    struct node * tmp;
    tmp = (struct node *) malloc ( size of (struct node));
    tmp → data = x;
    ~~tmp → next = NULL;~~
    if (start = = NULL)
      { start = tmp ;
        tmp → next = start;
      }
```

Creation of Circular Linked List

```
else {
struct node *tmp1 = start;
        while (tmp1->next != start)
        {
          tmp1 = tmp1->next;
        }
        tmp1->next = tmp;
        tmp->next = start;
}
```

Time Complexity of Creation of Circular linked list = O(n)

Traversal of Circular Linked List

```
void display()
{
  struct node *p;
    p = start;
  if(start == NULL)
  printf("linked list is empty");
  else
    { while(p->next != start)
      { printf("%d", p->data);
        p = p->next;
      }
  printf("%d", p->data);
  }
```

VIPS Vivekananda Institute of Professional Studies
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

Insertion at Beginning of Circular Linked List

```
void insertatbeg(int x)
{
  struct node *tmp;
  tmp = (struct node*)malloc(sizeof(struct node));
  tmp->data = x;
  tmp->next = NULL;
  if(start == NULL)
    { start = tmp;
      tmp->next = start;
    }
  else {
```

Insertion at Beginning of Circular Linked List

```
struct node *q = start;
while(q->next != start)
{
    q = q->next;
}
    tmp->next = start;
    q->next = tmp;
    start = tmp;
}
```

Time Complexity insertion at beg = O(n)

Insertion at End of Circular Linked List

```
void insert at end (int x)
{
 struct node *tmp;
 tmp = (struct node *) malloc (sizeof (struct node));
 tmp→data = x;
 tmp→next = NULL;
 if (start == NULL)
  {
     start = tmp;
     tmp → next = start;
  }
else
 { struct node *q = start;
        while (q→next != start)
```

Insertion at End of Circular Linked List

```
    {
        q = q->next;
    }
    q->next = t;
    t->next = start;
}
```

Time Complexity Insertion at end = O(n)

Deletion at Beginning of Circular Linked List

```
void deletionatbeg()
{
  struct node *p;
  if(start == HULL)
  printf("List is empty");
  else if
    {
      p = start;
      while(p -> next != start)
      {
        p = p -> next;
      }
      p -> next = start -> next;
        free(start);
      start  = p -> next;
    }
}
```

Time Complexity Deletion at beg = O(n)

Deletion at End of Circular Linked List

```
void deletion at end()
{
  struct node *p, *q;
 if (head == NULL)
  {
   printf("List is empty");
      return;
  }
  else
  {  p = start;
     while(p->next != start)
      {
       p=p->start;
      }
        q = p;
        p = p->next;
     }
     q->next = p->next;
          free(p);
          free(p);
     }}
```

END